



Cloud Readiness Check – M1
SAP Commerce Cloud
MTEL.me

LEGAL DISCLAIMER

The contents of this document shall remain the confidential property of SAP and may not be communicated to any other party without the prior written approval of SAP. This document must not be reproduced in whole or in part. It must not be used other than for evaluation purposes only by Customer except with the prior written consent of SAP and then only on condition that SAP's and any other copyright notices are included in such reproduction.

The furnishing of this document is subject to contract and shall not be construed as an offer or as constituting a binding agreement on the part of SAP to enter into any relationship. SAP provides this document as guidance only to estimate costs and timescales of the predicted delivery project. This will be subject to confirmation prior to any contractual or delivery commitment by SAP. Any services to be provided by SAP as proposed in this document shall be subject to the standard terms and conditions of SAP for the provision of the relevant services.

SAP has no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, referenced documents and SAP's strategy and possible future developments, products and /or platform directions and functionality are all subject to change and may be changed from SAP at any time for any reason and without notice. The information in this document and any referenced documents is provided without a warranty of any kind, either express or implied, including but not limited to, the implied warranties of merchantability, fitness for a particular purpose, or non-infringement. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Customer is cautioned not to place undue reliance on these forward-looking statements, and they should not be relied upon in making purchasing decisions.

Those who prepared this response have taken reasonable care in preparing it and believe its contents to be true as at the date of this document. SAP cannot warrant the truth of matters outside of its control and accordingly does not warrant the truth of all statements set out in this document to extent that such statements derive from facts and matters supplied by other persons to SAP. The statements in this document are qualified accordingly.

DOCUMENT REVISION HISTORY

Date	Revision	Author	Summary of Changes
01/02/2021	1.0	josef.vilimek@sap.com	

TABLE OF CONTENTS

INTRODUCTION	5
PROJECT SETUP	6
SAP Commerce Patch Level version Compatibility check.....	6
Commerce Cloud Default Configuration	6
Extensions and Dependencies	6
Configuration settings	7
Source Control	8
Impex Files.....	8
Patches	9
Storefront compression with Wro4j	9
Cloud Portal Configurations	9
CATALOG DESIGN	14
Product Catalog Design	14
Content Catalog Design	19
Multi-Tenancy	22
CODE REVIEW	23
Source Code Analysis.....	23
Data Model / Type System.....	26
Type System and Relational Data Base Management System (RDBMS).....	29
Third-Party Libraries	30
Build Customization	31
INTEGRATION OVERVIEW	32
NFRs	32
SUMMARY OF FINDINGS	33

INTRODUCTION

This document describes the Cloud Readiness Check (CRC) Milestone 1 results for MTel, conducted by SAP CX Field Services between 02.02.2021 and 09.02.201.

The CRC is executed in two phases – Milestones 1 and 2. Each milestone has different objectives and is executed in different phases of the project. This report contains the findings for Milestone 1, which includes a preliminary analysis of the solution's design, set-up and codebase. It provides recommendations to reduce the risks when deploying to the SAP Commerce Cloud.

It is intended as a check for Cloud Service to be confident that your solution is of the required level of quality before it goes into production. It does not include system nor application fixes as a result of the review performed. Any actions recommended as a result of the Cloud Readiness Check, will have to be accessed and taken by delivery team.

If a in-depth review is required, we encourage you to further explore how review and validation services offered by SAP Customer Experience Services can help to further boost the quality of your project. Please contact your Customer Engagement Manager for more details.

Activities

The following tasks were planned and executed between 02.02.2021 and 09.02.201:

- Static Source Code Analysis
- Catalog Design Review
- Type System Customizations (Data Model)
- Project Setup

Each chapter in this report contains detailed information on a specific topic, highlighting issues that need to be addressed and providing recommendations on potential improvements. These activities are in line with the recommended practices collected in SAP CX Field Services methodology. Further information can be found in the official documentation: <https://www.sap.com/cxworks/>.

Important Recommendations

This review identified that MTel is generally **following** the SAP recommended practices for project implementation. The following observations are considered of importance:

1. Some property configuration need review (tomcat.generaloptions is cloud managed property should not be explicitly declared for cloud environments)
2. Few Impex files are not fully imported during initialization
3. Several endpoints are exposed to public internet (API, Backoffice, Solr)
4. 2 bugs found in the source code analysis
5. 17 medium and 5 low priority issues were found in the type system setup
6. One problem with HYBRIS_LONG_STRING data type was found

Next Steps

Once the recommendations of this report have been evaluated and, if necessary, implemented, Milestone #2 should be prepared for and planned:

1. Review the recommendations from this report;
2. Prepare for Milestone #2.

PROJECT SETUP

Overview

This section reviews the overall setup of the *mtel* project, including Extensions and AddOns and source control setup. These items are important to the overall health and success of a project.

SAP Commerce Patch Level version Compatibility check

PS-01

Project is using SAP Commerce version 2005 with the auto-patching feature enabled.

PS-02

The current SAP Commerce SOLR setup is supported by SAP Commerce Cloud and is in line with the used SAP Commerce compatibility matrix. Solr version deployed is 8.4.1.

Commerce Cloud Default Configuration

Commerce Cloud has pre-defined values particular nodes. List of managed properties is available on SAP Help: <https://help.sap.com/viewer/1be46286b36a4aa48205be5a96240672/SHIP/en-US/a30160b786b545959184898b51c737fa.html>

PS-03

tomcat.generaloptions property is explicitly declared in common/local.properties file. This is the managed property in cloud and during the build process is overwritten.

Extensions and Dependencies

The following code block is the current setup, as found in **localextensions.xml** file.

```

<hybrisconfig xmlns:xsi='http://www.w3.org/2001/XMLSchema-instance'
xsi:noNamespaceSchemaLocation='../bin/platform/resources/schemas/extensions.xsd'
>
  <extensions>
    <path dir='${HYBRIS_BIN_DIR}' autoload='false' />
    <extension name='b2ctelcobackoffice' />
    <extension name='b2ctelcowebseervices' />
    <extension name='b2ctelcoservices' />
    <extension name="adaptiveSearchbackoffice" />
    <extension name="adaptiveSearchsolr" />
    <extension name='commerceservicesbackoffice' />
    <extension name='solrserver' />
    <extension name='subscriptionbackoffice' />
    <extension name='cmswebseervices' />
    <extension name='customersupportbackoffice' />
    <extension name='acceleratorservices' />
    <extension name='ldap' />
    <extension name="patchesbackoffice"/>

    <extension name="mtmcore"/>
    <extension name="mtminitialdata"/>
    <extension name="mtmfacades"/>
    <extension name="mtmtest"/>
    <extension name="mtmbackoffice"/>
    <extension name="mtmcommercewebseervices"/>
    <extension name="mtmmockserver"/>
    <extension name="mtmintegration"/>
    <extension name="mtmpatches"/>

  </extensions>
</hybrisconfig>

```

PS-04

ldap extension is marked as deprecated since Commerce version 6.1. At the moment there is no suitable replacement available hence this extension remains in 2005 SAP commerce version.

Configuration settings

It's recommended to maintain distinct configurations between different application server roles, follow SAP CX Field Services recommended practices and define specific configurations for front-end and back-end application servers.

1. The different configurations for development, staging and production have to be stored correctly in the source control
2. All settings have to be reviewed for correct usage in the dedicated environment and documented accordingly

PS-05

Current configuration settings are correct and in line with best practices.

Source Control

It is not enough to simply have a source control system in place to track changes to the source code. The source control system must be used correctly. Files generated by the SAP Commerce build process should not be committed to the repository; they should be ignored.

In addition, it is important to use a well-defined branch and tagging strategy to ensure that developers can work in parallel with minimal impact on each other. A well-defined branch and tagging strategy also makes it easier to manage, maintain, and support both internal and external releases.

There are a number of concerns that should always be satisfied through the use of source control:

- The current project should make use of tags and branching. In a collaborative project using Git, the recommendation is to use feature branches for development and tagging to track versions.
- Source code comments should be linked to a work-item reference (e.g. Jira ticket). This assists in terms of tracking why changes have been made and for what.
- Pull requests should always be created before merging new code. It is recommended to review, discuss, and approve code changes related to the pull requests before merging any-feature branch to master.

PS-06

There is an SCM in use, with proper configuration and usage. SCM contains detailed instructions on how to build the local instance.

https://bitbucket.org/Wireless_Media/mtelme-hybris/src/dev/

Impex Files

These files are loaded during platform initialization and updates. In a production environment Impex files are usually responsible for core configuration and initial data. The objective of this evaluation is to check if these files are being properly loaded, without any warnings or errors.

PS-07

Few errors while importing impex files during initialization:

- 1) [ImpExSystemSetup] importing resource : /impex/essentialdataAdhocWorkflowLocalizations_XX.impex
- 2) [ImpExSystemSetup] importing resource : /customersupportbackoffice/impex/essentialdata-customersupportgroups_XX.impex

Exception when Executing indexer worker:

ERROR: [doc=8796099608577] unknown field '_backofficeLabel_text_me'

Patches

PS-07

During the review we did not find the use of the ootb patching framework: <https://help.sap.com/viewer/d0224eca81e249cb821f2cdf45a82ace/1808/en-US/f5394169c3534d548839aec9f7bc3550.html>.

Storefront compression with Wro4j

Commerce Accelerator storefronts use wro4j for compressing CSS style files and JavaScript source files. It is very important to make sure that Accelerator-based storefronts have this feature enabled as it affects performance to a great extent.

PS-08

WRO4J storefront compression is not enabled. But that is irrelevant as SAP Commerce is used in headless mode.

Cloud Portal Configurations

In the Cloud Portal it is possible to configure the following properties:

Property	Description	Value																																
Endpoints	Virtual hosts that can be mapped onto applications running on CCv2. They are automatically allocated a DNS name.	P1 standard set of endpoints configured Public Endpoints <table><thead><tr><th>Name</th><th>SSL Proxy</th><th>URL</th><th>Service</th></tr></thead><tbody><tr><td>API</td><td>Default</td><td> api-city/yfcln-mtldoa01-p1-public.mst.API</td><td></td></tr><tr><td>Backoffice</td><td>Default</td><td> backoffice-city/yfcln-mtldoa01-p1-public.Backoffice</td><td></td></tr><tr><td>JS Storefront</td><td>Default</td><td> jsapp-city/yfcln-mtldoa01-p1-public...JS Storefront</td><td></td></tr><tr><td>Logging</td><td>Default</td><td> logs-city/yfcln-mtldoa01-p1-public.ms.logging@hana</td><td></td></tr><tr><td>Background Processing</td><td>Default</td><td> backgroundprocessing-city/yfcln-mtldoa01-p1-public.backgroundprocessing</td><td></td></tr><tr><td>Solr</td><td>Default</td><td> solr-city/yfcln-mtldoa01-p1-public.mst.Solr</td><td></td></tr><tr><td>Storefront</td><td>Default</td><td> storefront-city/yfcln-mtldoa01-p1-pa.Storefront</td><td></td></tr></tbody></table>	Name	SSL Proxy	URL	Service	API	Default	api-city/yfcln-mtldoa01-p1-public.mst.API		Backoffice	Default	backoffice-city/yfcln-mtldoa01-p1-public.Backoffice		JS Storefront	Default	jsapp-city/yfcln-mtldoa01-p1-public...JS Storefront		Logging	Default	logs-city/yfcln-mtldoa01-p1-public.ms.logging@hana		Background Processing	Default	backgroundprocessing-city/yfcln-mtldoa01-p1-public.backgroundprocessing		Solr	Default	solr-city/yfcln-mtldoa01-p1-public.mst.Solr		Storefront	Default	storefront-city/yfcln-mtldoa01-p1-pa.Storefront	
Name	SSL Proxy	URL	Service																															
API	Default	api-city/yfcln-mtldoa01-p1-public.mst.API																																
Backoffice	Default	backoffice-city/yfcln-mtldoa01-p1-public.Backoffice																																
JS Storefront	Default	jsapp-city/yfcln-mtldoa01-p1-public...JS Storefront																																
Logging	Default	logs-city/yfcln-mtldoa01-p1-public.ms.logging@hana																																
Background Processing	Default	backgroundprocessing-city/yfcln-mtldoa01-p1-public.backgroundprocessing																																
Solr	Default	solr-city/yfcln-mtldoa01-p1-public.mst.Solr																																
Storefront	Default	storefront-city/yfcln-mtldoa01-p1-pa.Storefront																																
SSL Certificates	Can be assigned to an endpoint.	All endpoints are configured with the default Let's encrypt (rotating) certificates.																																
IP Filters	Can be defined for an endpoint, ensuring that only traffic from certain IP addresses can access an endpoint.	API - allow all Backoffice - allow all JS Storefront - deny all Background processing - deny all Solr - allow all Storefront - allow all																																

Redirects	Can be defined for an endpoint, enabling vanity & marketing URLs and site migrations.	❓ No redirects configured
Static Artifacts	Can be uploaded for an endpoint, enabling search engines to better understand the site.	❓ No static files configured
Maintenance pages	Can be uploaded and enabled for an endpoint, to provide a friendly message during scheduled maintenance periods.	❓ No maintenance page configured

PS-09

Backoffice, API and Solr endpoints are exposed to public internet. It's recommended to use IP filter instead to avoid security risks.

Manifest.json

```
{
  "commerceSuiteVersion": "2005",
  "disableImageReuse": false,
  "extensionPacks": [
    {
      "name": "hybris-b2ctelco-suite",
      "version": "2007.0"
    }
  ],
  "extensions": [
    "backoffice"
  ],
  "useConfig": {
    "properties": [
      {
        "location": "build/config/common/local.properties"
      },
      {
        "location": "build/config/dev/local.properties",
        "persona": "development"
      },
      {
        "location": "build/config/stage/local.properties",
        "persona": "staging"
      },
      {
        "location": "build/config/prod/local.properties",
        "persona": "production"
      }
    ]
  },
  "extensions": {
    "location": "build/config/common/localextensions.xml",
    "exclude": [
      "mtmtest"
    ]
  }
},
```

```

    "solr": {
      "location": "core-customize/solr-conf"
    }
  },
  "aspects": [
    {
      "name": "backoffice",
      "webapps": [
        {
          "name": "hac",
          "contextPath": "/hac"
        },
        {
          "name": "mediaweb",
          "contextPath": "/medias"
        },
        {
          "name": "backoffice",
          "contextPath": "/backoffice"
        }
      ]
    },
    {
      "name": "backgroundProcessing",
      "webapps": [
        {
          "name": "hac",
          "contextPath": "/hac"
        },
        {
          "name": "mediaweb",
          "contextPath": "/medias"
        }
      ]
    },
    {
      "name": "api",
      "properties": [
        {
          "key": "xss.filter.header.X-Frame-Options",
          "value": ""
        }
      ]
    },
    {
      "name": "webapps": [
        {
          "name": "mtmcommercewebservices",
          "contextPath": "/mtmcommercewebservices"
        },
        {
          "name": "oauth2",
          "contextPath": "/authorizationserver"
        },
        {
          "name": "cmswebservices",
          "contextPath": "/cmswebservices"
        }
      ]
    }
  ]
}

```

```

    },
    {
      "name": "permissionswebservice",
      "contextPath": "/permissionswebservice"
    },
    {
      "name": "mediaweb",
      "contextPath": "/medias"
    },
    {
      "name": "mtmockserver",
      "contextPath": "/mtmockserver"
    },
    {
      "name": "hac",
      "contextPath": "/hac"
    }
  ]
},
{
  "name": "accstorefront",
  "webapps": [
    {
      "name": "mediaweb",
      "contextPath": "/medias"
    }
  ]
}
]
}

```

Code Block 1 manifest.json

Property	Description	Value	Comments
commerceSuiteVersion	The exact version of the Commerce Suite artefact to download. Recommendation is to not include the patch number explicitly.	2005	✓ auto patching enabled
extensions	The list of extensions to be enabled	empty	✓
extensionPacks	The extension-packs section contains a list of named and versioned artifacts	hybris-b2ctelco-suite 2007.0	✓
storefrontAddons	The AddOns installed by the Commerce Platform during the build process.	empty	✓
aspects	The aspect-specific configuration.	• backoffice • backgroundProcessing • api • accstorefront	✓

useConfig	Allows the reuse of the configuration available in the repository with a customer project. It consists of extensions, properties, Solr configuration, and languages.	<u>common</u> build/config/common/local.properties <u>development persona</u> build/config/dev/local.properties <u>staging persona</u> build/config/stage/local.properties <u>production persona</u> build/config/prod/local.properties <u>extensions</u> build/config/common/localextensions.xml	✓
tests	This section is used to configure extension tests to run during the build process	<i>empty</i>	✓
webTests	This section is used to configure webTests to run during the build process	<i>empty</i>	✓
properties	A list of common properties to be set in the local properties for all aspects (key-value pair)	<i>empty</i>	✓

PS-10

This review identified that the manifest.json definition is correct and contains all of the expected configurations.

1. commerceSuiteVersion is configured to use the latest patch instead of explicitly defining a patch.
2. Configurations have been reused to make local development as close to what will be deployed in production.

CATALOG DESIGN

Overview

This section of the review is an analysis of mtel's catalog design.

Background

Catalogs are typically used to logically group items (products, categories, media) that have a similar purpose. For example, catalogs can be used to group products or WCMS components by region, brand, and/or channel. In addition to the benefits they provide to organizing and grouping items, catalogs also have the added benefit of creating a hierarchical structure in which upstream catalogs can override data in downstream catalogs.

CatalogVersions are assigned to a catalog. Each catalog can have multiple CatalogVersions assigned, but only one active CatalogVersion. It is not mandatory to have more than a single CatalogVersion for a Catalog, but the most common setup is to have a *Staged* and *Online* CatalogVersion. Staged is the working CatalogVersion where users maintain the data and then eventually publish to the Online or active CatalogVersion.

Catalog design is a very important part of the solution architecture because a design that doesn't follow best practices can cause performance, scalability, and maintenance issues due to the fact that each CatalogVersion within a Catalog is an entire copy of the items within the Catalog.

When evaluating the Product Catalog design the following should be considered:

- Business requirements
 - e.g. Do the product assortments differ depending on region?
- Maintainability
 - Who is going to be responsible for maintaining the data?
 - Do business users in each region maintain their own data?
- Data volumes
 - Performance and scalability considerations.

Product Catalog Design

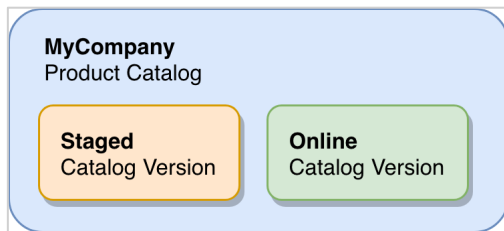
Eventually it's time to discuss mtel's product catalog design approach, but first to cover the three SAP-recognized approaches for designing a product catalog structure. Each has strengths and weaknesses, and the choice depends on a project's requirements. Covering the SAP-recognized approaches first helps to comprehend the approach taken by mtel.

SAP-Recommended Product Catalog Designs

The three SAP-recognized approaches are:

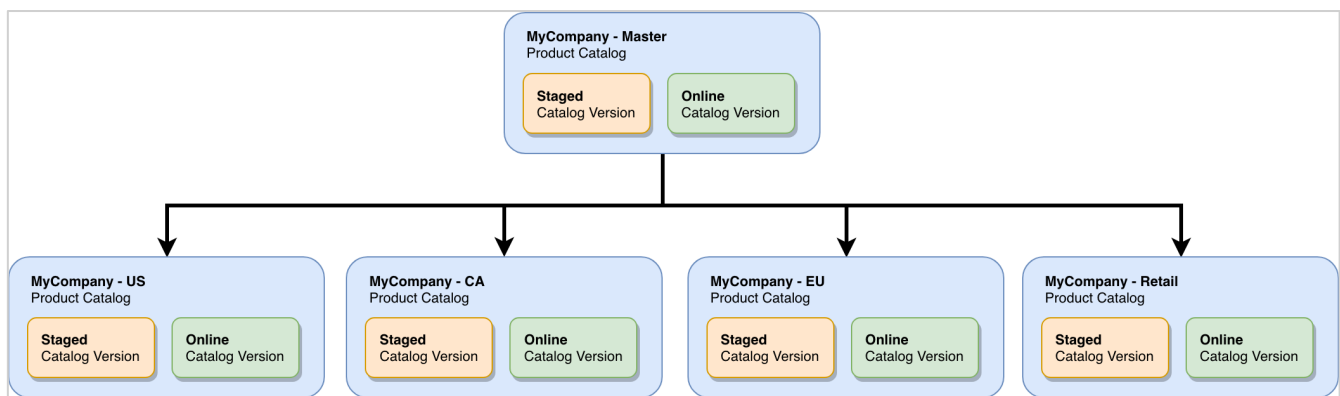
1. Single Product Catalog
2. Master Product Catalog with Downstream Catalogs
3. Single Product Catalog with Filtering

1. Single Product Catalog



This is the simplest catalog design. We see it used on many implementations. It has a single product catalog containing two catalog versions: "staged" and "online". This design assumes that the business has either a single channel, region, or brand and/or that it is using an alternative and custom way of dealing with the typical use cases for multiple Catalogs.

2. Master Product Catalog with Downstream Catalogs



This is the most common approach when business requirements call for different assortments of products (e.g. different assortments depending on region or selling channel). In this design approach, product catalogs fall into one of the two groups:

Master Catalog

As its name implies, this is the parent catalog that contains all products. If products are imported from another source system, this is the catalog where the products will be imported into. This catalog can be used for aggregating multiple sources of data to make a complete product record before publishing to the downstream catalogs. Some business users will likely be managing and enriching the product content in this catalog. Restrictions on which business users can read or write in this catalog can be defined using catalog permissions.

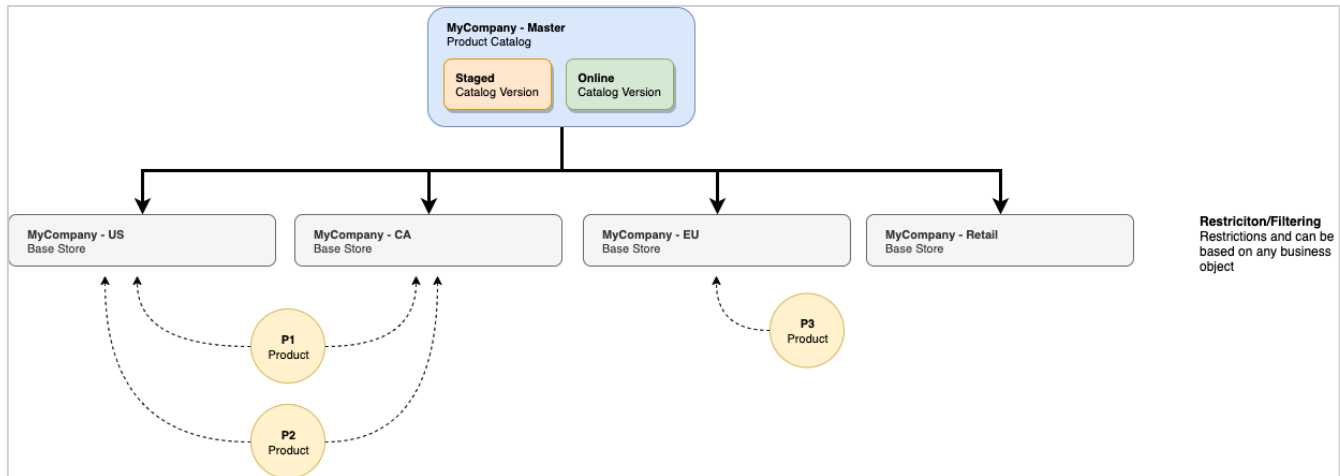
Downstream Catalogs

These are the child catalogs that contain a subset of the master catalog's products. With this design, each downstream catalog represents a different assortment of products. For example, downstream catalogs may represent different regions, selling channels, brands or seasons. Each downstream catalog typically has two catalog version "staged" and "online". Business users manage and override inherited products in the "staged" catalog version. Prices are typically managed in these downstream catalogs because

managing them in the master catalog may not give the flexibility needed to set a price per region, channel, brand or season.

The approach works well when product catalog management is decentralized (e.g. German users manage the DE Catalog, French users manage the FR Catalog). However, having a separate catalog for each region, channel, brand, etc. causes significant overhead; the system needs to store and index multiple copies of catalog data and users also need to manage multiple copies of catalog data. For instance, if a product attribute changes, users may need to modify each catalog's copy of that product.

3. Single Product Catalog with Filtering



If the requirement is to simply have different catalog assortments from a super set of products and this should be done in the most efficient way to streamline the back-end processes by reducing the number of product records, then there is an alternative design that can be used. This requires a more complex customization compared to using multiple catalogs. However, the performance and scalability benefits often outweigh the extra effort.

This approach works by defining a relation between the product type and another SAP Commerce type that represents what we would like our product assortments to be based on. BaseStore is usually a good choice because it can generically represent a country, region, channel, brand or any combination of these. It is possible to use country or a custom type to define the owner of the assortment. The relation can define an inclusive or exclusive set of products depending on which is easier for the business to manage. For example, if most assortments are the same and it's only needed to exclude a small set of products from some assortments, then it would probably make sense to define an exclusion set of products when defining the relation.

Points to be considered for this approach:

- It can be a significant customisation, but SAP CX Field Services has a [Solution Add-On](#) called **Multi Country and Channel Product Enhancements** that can be integrated into a project with minimal effort.
- Less autonomy for business users when product management is decentralised (e.g. regional product managers will not be able to customise their products).
- There is one target product to sync which means that all country/brand product managers must be aligned when they want to publish a product.
- Less options for overriding content e.g. in Switzerland it might be a requirement to override some of the texts in the French content. To achieve this we have to add another language (fr_CH).

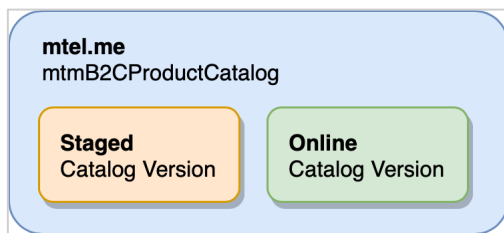
SAP provides guidelines and recommendations related product and content catalog design available in online documentation:

[Product Catalog](#)

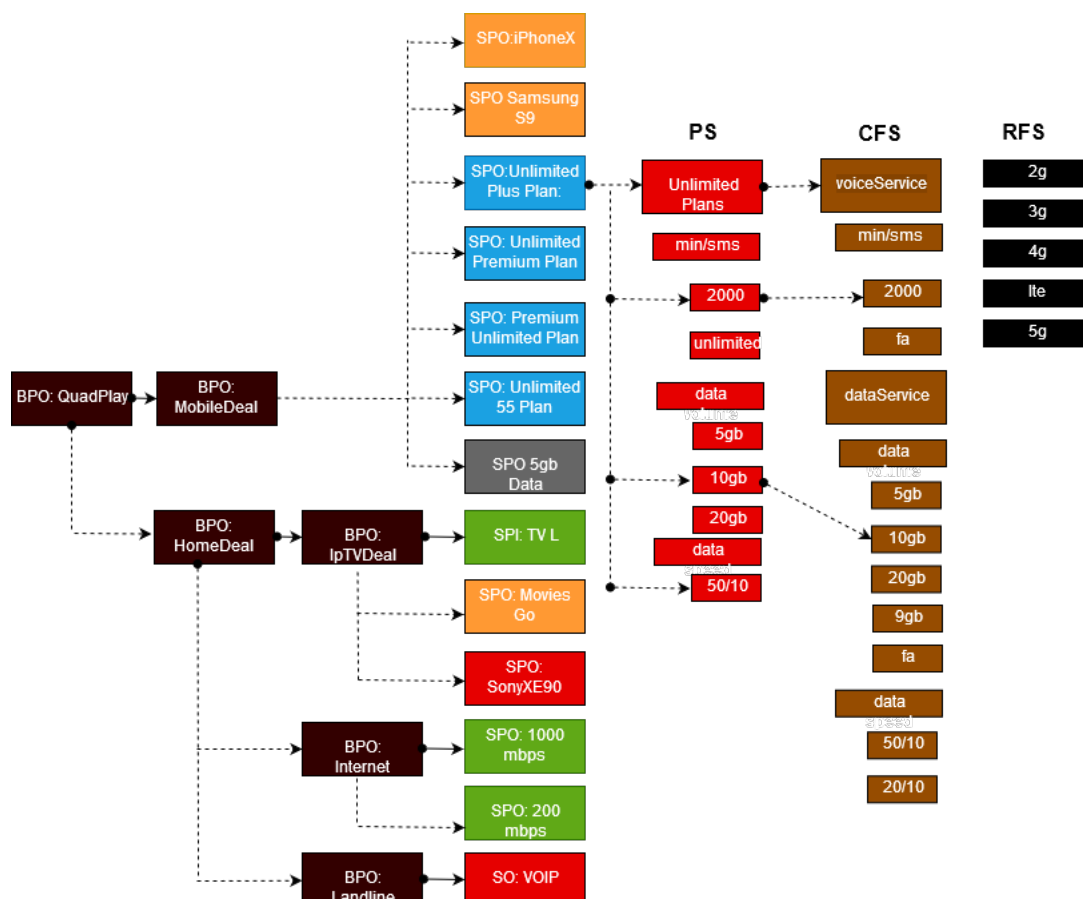
[https://www.sap.com/cxworks/article/441918448/Strategies for Designing your SAP Commerce Cloud Content Catalogs](https://www.sap.com/cxworks/article/441918448/Strategies%20for%20Designing%20your%20SAP%20Commerce%20Cloud%20Content%20Catalogs)

[https://www.sap.com/cxworks/article/441914544/Implementing Internationalization with SAP Commerce Cloud](https://www.sap.com/cxworks/article/441914544/Implementing%20Internationalization%20with%20SAP%20Commerce%20Cloud)

Product Catalog Design



On Selfcare portal only Packages and Addons for different services (mobile, tv, internet, fixed) are offered. Stock is not being used, so there are no SKU's, variants and so on. Project is leveraging the Telco accelerator for that purpose.



CD-01

The current catalog design is implemented based on SAP Commerce catalog design #1 (Single Product Catalog) but following the data model from Telco accelerator

Product Variants

SAP Commerce offers two out of the box solutions for modelling variant products:

1. The style and size VariantProduct implementation used by B2C accelerator.
2. The multidimensional generic variants of the B2B accelerator.

CD-02

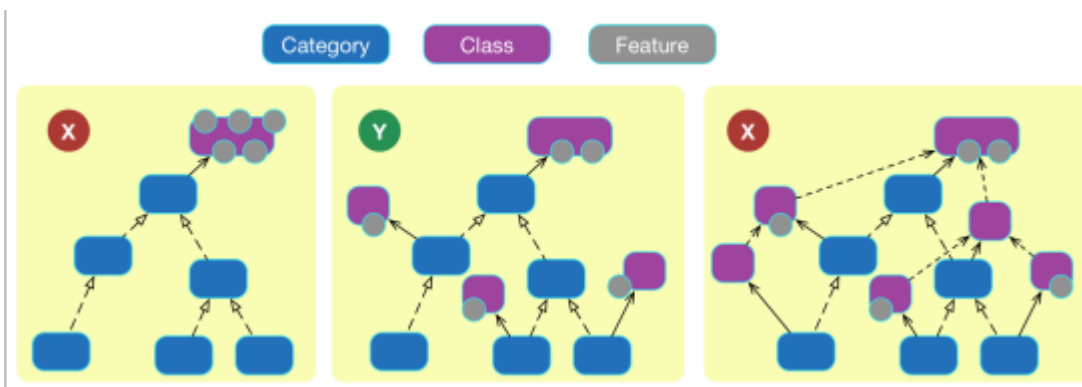
Product variants are made following SAP CX Field Services recommended practices.

Classification System

The recommended practice for the Classification structure is to have the structure as simple as possible. The ideal design typically sits somewhere in between these two scenarios:

- Declare a Classification Class only if a feature needs to be assigned.
- Do not declare parent-child relationships on the Classification Classes if the same relationship is declared via Category structure.

The picture below illustrates a clear and healthy scenario in the middle, however the two on the sides could lead to performance problems.



Moreover it's recommended to introduce a good naming convention, especially if the Classification System is maintained manually. The business user needs to be able to easily understand what is a regular Category used for marketing or navigation purposes, and what is a Classification Class used for defining features on Products.

For this project, the following Classification structure is in place: classification system is not used.

CD-03

The Classification System is not used.

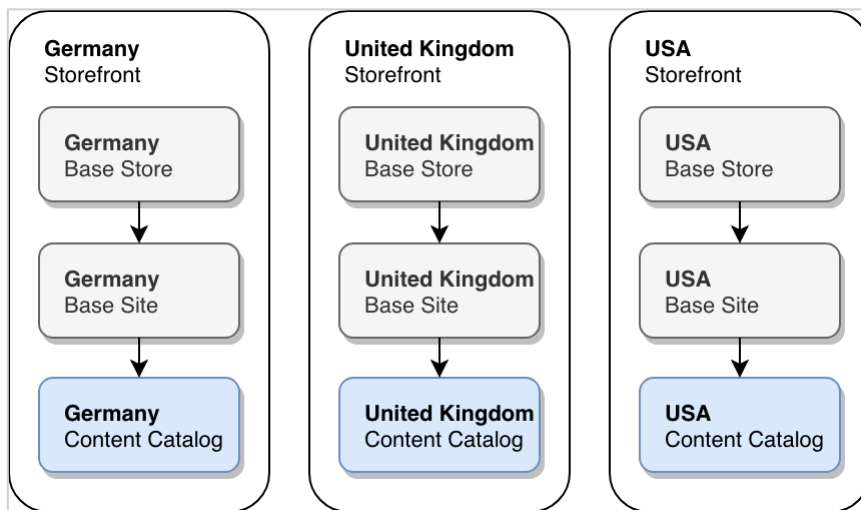
Content Catalog Design

Once again, we first cover the three SAP-recognized approaches for designing a content catalog structure before discussing mtel's approach. Each approach has strengths and weaknesses, and the choice depends on a project's requirements. Covering the SAP-recognized approaches first helps to comprehend the approach taken by mtel.

The three SAP-recognized approaches are:

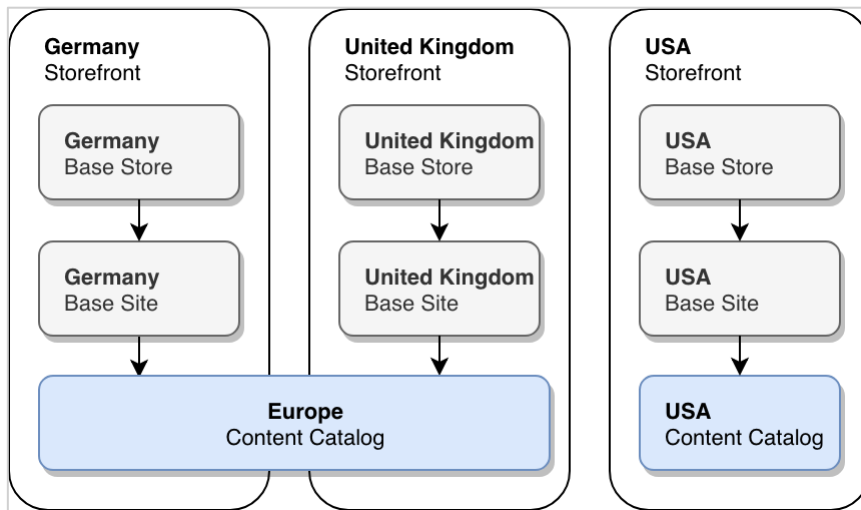
1. **Single Content Catalogs**
2. **Shared Content Catalog**
3. **Multi-layered Content Catalog**

1. Single Content Catalogs



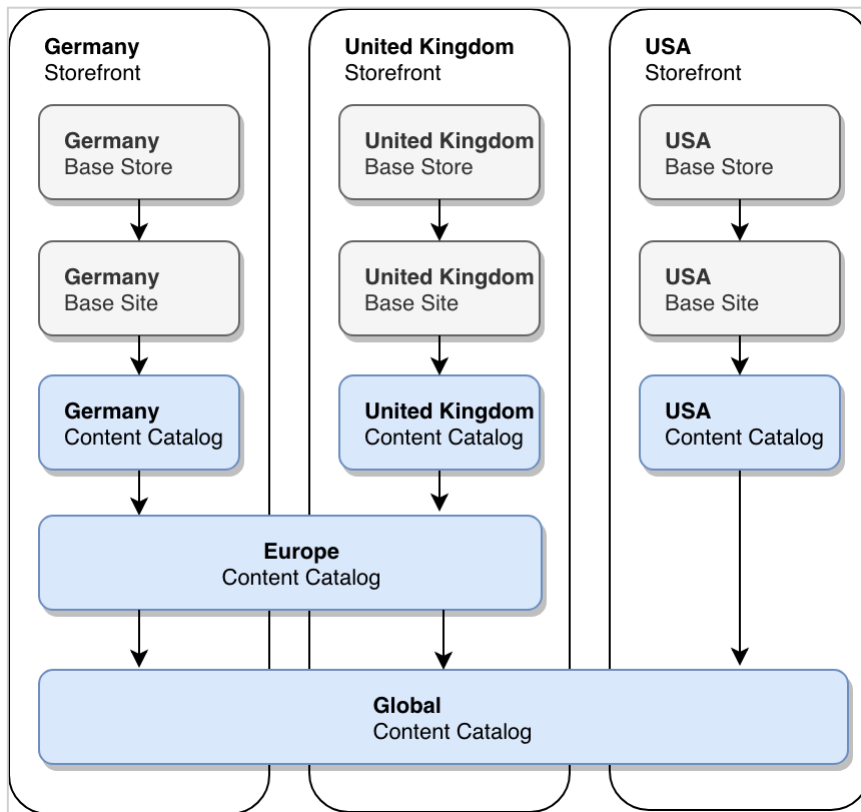
In this approach, each storefront has its own separate content catalog. This approach is used in the accelerator storefronts. In this setup, there is no ability to reuse content between the different storefronts. This approach is well-suited for organizations that want to decentralize the management of their content. This approach is not well-suited for reusing content between different storefronts; templates cannot be shared across stores.

2. Shared Content Catalog



In the second approach, some storefronts share content catalogs. This design can be configured and is supported in the standard WCMS module. Unlike the first approach, content can be managed and shared from a central place. In order to differentiate content for different storefronts connected to the same content catalog (e.g. a page or component that is only visible for DE) can the standard [WCMS restriction system](#) be used. The restriction system dictates whether pages or components are visible to users. Since there is no OOTB restriction for storefronts, it requires a small customization, so that content can be restricted to be displayed when it is loaded in a specific storefront.

3. Multi-layered Content Catalog



The third approach is a mixture of the first two approaches. Like in the first approach, each storefront has its own separate content catalog, but also has additional shared content catalogs like in the second approach. This approach requires customizations to the standard WCMS module. This approach is popular for organizations that are active in multiple countries. These organizations would have at a minimum a global and local layer where content is being created. The central organization typically would create and maintain templates, common pages and content, like the checkout pages or organization pages. The local department would maintain the homepage, landing pages and campaign content. In some organizations, there is the requirement for additional layers (e.g. regional layers).

With the release of the SAP Commerce Solution version 6.5 multi-country support for WCMS is built into SmartEdit.

Content Catalog Design

CD-04

Standard one content catalog approach with two catalog versions. MTel is using SAP Commerce in headless mode.

Multi-Tenancy

SAP Commerce can be run in a multi-tenant mode. In this mode several distinct sets of data are maintained on one single SAP Commerce installation. The effect is that you can host several online shops for different customers using one SAP Commerce installation.

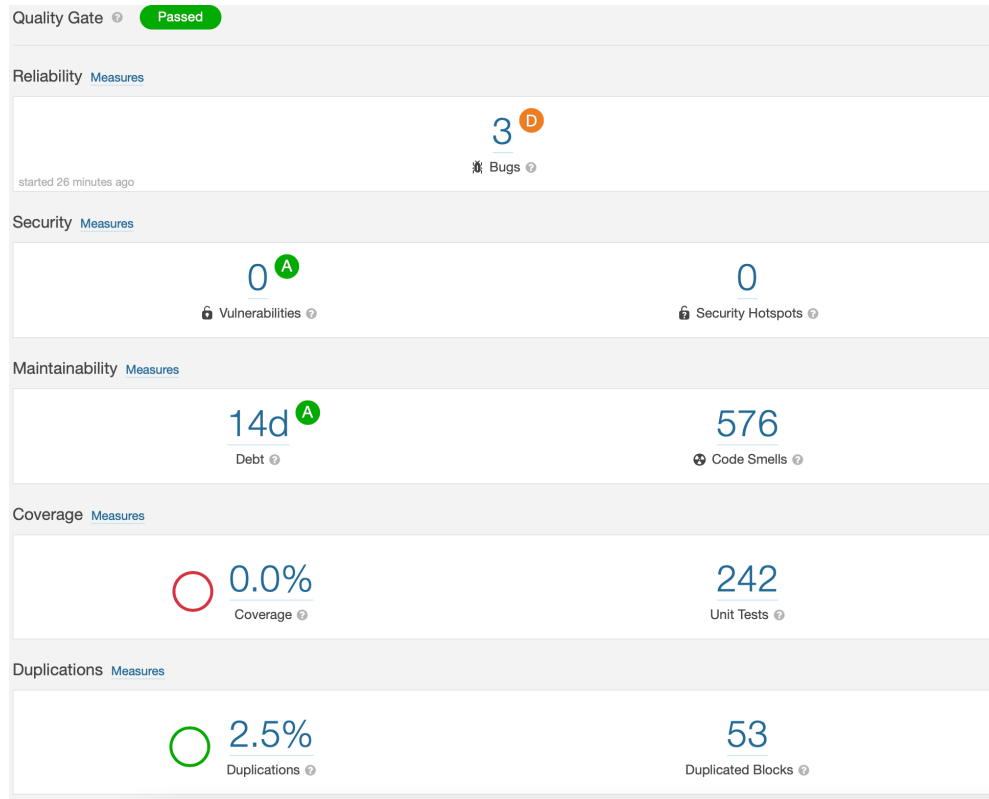
CD-05

The Multi-tenancy mode is inactive and follows the SAP CX Field Services recommended practices.

CODE REVIEW

Source Code Analysis

This is the Sonar Overview after executing a full static analysis with the SAP CX Field Services ruleset.



Bugs & Vulnerabilities

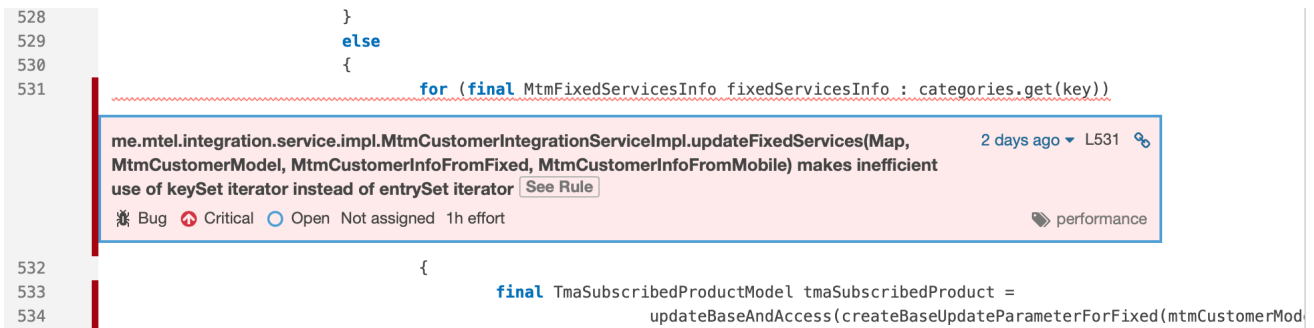
CR-01

Only two bugs (1 critical, 1 major + 1 false positive) were found by the Sonarqube.

1) *hybris/.../src/me/mtel/core/service/impl/MtmExcelImportServiceImpl.java, line 80*

"BigDecimal(double)" should not be used rule - this seems to be a false positive finding.

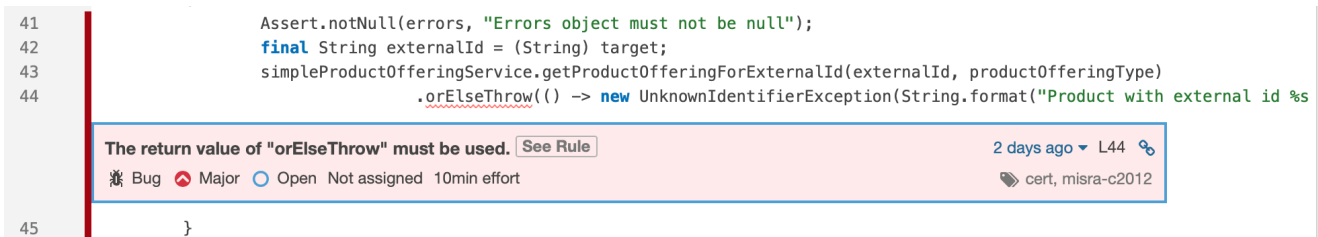
2) *hybris/.../me/mtel/integration/service/impl/MtmCustomerIntegrationServiceImpl.java, line 531*



me.mtel.integration.service.impl.MtmCustomerIntegrationServiceImpl.updateFixedServices(Map, MtmCustomerModel, MtmCustomerInfoFromFixed, MtmCustomerInfoFromMobile) makes inefficient use of keySet iterator instead of entrySet iterator

This method accesses the value of a Map entry, using a key that was retrieved from a keySet iterator. It is more efficient to use an iterator on the entrySet of the map, to avoid the Map.get(key) lookup.

3) *hybris/.../web/src/me/mtel/validator/MtmProductOfferingExternalIdValidator.java, line 44*



Return values from functions without side effects should not be ignored

- Bug
- Major
- Main sources
- cert, misra-c2012
- Available Since Feb 09, 2021
- SonarAnalyzer (Java)
- Constant/issue: 10min

When the call to a function doesn't have any side effects, what is the point of making the call if the results are ignored? In such case, either the function call is useless and should be dropped or the source code doesn't behave as expected.

To prevent generating any false-positives, this rule triggers an issue only on the following predefined list of immutable classes in the Java API :

- java.lang.String
- java.lang.Boolean
- java.lang.Integer
- java.lang.Double
- java.lang.Float
- java.lang.Byte

- java.lang.Character
- java.lang.Short
- java.lang.StackTraceElement
- java.time.DayOfWeek
- java.time.Duration
- java.time.Instant
- java.time.LocalDate
- java.time.LocalDateTime
- java.time.LocalTime
- java.time.Month
- java.time.MonthDay
- java.time.OffsetDateTime
- java.time.OffsetTime
- java.time.Period
- java.time.Year
- java.time.YearMonth
- java.time.ZonedDateTime
- java.math.BigInteger
- java.math.BigDecimal
- java.util.Optional

Noncompliant Code Example

```
public void handle(String command){
    command.toLowerCase(); // Noncompliant; result of method thrown away
    ...
}
```

Compliant Solution

```
public void handle(String command){
    String formattedCommand = command.toLowerCase();
    ...
}
```

Exceptions

This rule will not raise an issue when both these conditions are met:

- The method call is in a try block with an associated catch clause.
- The method name starts with "parse", "format", "decode" or "valueOf" or the method is String.getBytes(Charset).

```
private boolean textIsInteger(String textToCheck) {

    try {
        Integer.parseInt(textToCheck, 10); // OK
        return true;
    } catch (NumberFormatException ignored) {
```

```

    return false;
}
}

```

Coverage

Much of the Accelerator code is not covered by tests, which is to be expected. As for the custom code, the following coverage was found:

CR-02

It was not possible to gather code coverage information due to technical issue with the setup.

Data Model / Type System

The following custom types were defined (or extended) in this project:

Code	Extends	Typecode	Generate	Autogenerate	Description
AbstractOrder			FALSE	FALSE	
Product			FALSE	FALSE	
TmaSimpleProductOffering			FALSE	FALSE	
MtmTmaProductSpecCharacteristicGroup	GenericItem	20003			Group for TmaProductSpecCharacteristics
MtmTmaProductSpecCharacteristicCatalogUnawareGroup	MtmTmaProductSpecCharacteristicGroup				
TmaProductSpecCharacteristicValue			FALSE	FALSE	
TmaSubscribedProduct			FALSE	FALSE	
MtmDeviceInfo	GenericItem	20005			
ProductReference			FALSE	FALSE	
MtmEmailRequest		20009			Email change request
PaymentTransaction			FALSE	FALSE	
PaymentTransactionEntry			FALSE	FALSE	
CreditCardPaymentInfo			FALSE	FALSE	
MtmCity	GenericItem	20010			
MtmForgottenPasswordProcess	ForgottenPasswordProcess		TRUE	TRUE	
MtmRegisterAccountProcess	StoreFrontProcess				
MtmChangeEmailRequestProcess	StoreFrontCustomerProcess				
MtmActivateIdleProcess	StoreFrontCustomerProcess		TRUE	TRUE	
MtmFailedPaymentConfirmProcess	StoreFrontCustomerProcess		TRUE	TRUE	
MtmSecurityCode		20007			
PaginatedCronJob	CronJob				
MtmLogOutCustomersCronJob	PaginatedCronJob				

Type System Validator

The Type System Validation (TSV) Tool is a tool created by SAP CX Field Services that is embedded in Sonar and evaluates custom types in the solution. The following items in the type system must be investigated:

FILENAME	LINE #	RULE	PRIORITY	ELEMENT	DESCRIPTION
mtmcore-items.xml	224	TypeHasAtLeastOneIndexSpecified	M	MtmGenericRegistration	Not at least one database indexes is defined for type.
mtmcore-items.xml	873	TypeHasAtLeastOneIndexSpecified	M	MtmSecurityCode	Not at least one database indexes is defined for type.
mtmcore-items.xml	172	NoDatabaseIndexesForUniqueAttributesOfType	M	MtmCustomer	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	197	NoDatabaseIndexesForUniqueAttributesOfType	M	Customer	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	250	NoDatabaseIndexesForUniqueAttributesOfType	M	MtmRegistration	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	679	NoDatabaseIndexesForUniqueAttributesOfType	M	ProductReference	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	873	NoDatabaseIndexesForUniqueAttributesOfType	M	MtmSecurityCode	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	907	NoDatabaseIndexesForUniqueAttributesOfType	M	PaginatedCronJob	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	919	NoDatabaseIndexesForUniqueAttributesOfType	M	MtmLogOutCustomersCronJob	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmcore-items.xml	224	NoUniqueAttributesDefinedForType	M	MtmGenericRegistration	No unique attributes defined for type
mtmcore-items.xml	720	UniqueIndexUsingNonUniqueAttributes	M	PaymentTransaction	A unique index should contain only unique attributes cause it limits duplicate inserts to be not possible in that combination.
mtmcore-items.xml	752	UniqueIndexUsingNonUniqueAttributes	M	PaymentTransactionEntry	A unique index should contain only unique attributes cause it limits duplicate inserts to be not possible in that combination.

mtmintegrati on-items.xml	34	PersistenceColumnTypeMustBeForAllSupportedDatabases	M	message	If a columntype tag is specified for a an attribute's persistence, it has to be defined for all supported databases(oracle,mysql,sqlserver,hsqldb,sap).
mtmintegrati on-items.xml	70	PersistenceColumnTypeMustBeForAllSupportedDatabases	M	url	If a columntype tag is specified for a an attribute's persistence, it has to be defined for all supported databases(oracle,mysql,sqlserver,hsqldb,sap).
mtmintegrati on-items.xml	23	TypeHasAtLeastOneIndexSpecified	M	DatabaseLogEntry	Not at least one database indexes is defined for type.
mtmintegrati on-items.xml	14	NoDatabaseIndexesForUniqueAttributesOfType	M	UpdateCustomerInfoCronJob	No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.
mtmintegrati on-items.xml	23	NoUniqueAttributesDefinedForType	M	DatabaseLogEntry	No unique attributes defined for type
mtmcore- items.xml	115	OrderingOfRelationShouldBeAvoided	L	Customer2MtmCustomer.mtmCustomers	Any side of a relation that has cardinality='many' should not have ordered='true' unless absolutely necessary
mtmcore- items.xml	126	OrderingOfRelationShouldBeAvoided	L	MtmTmaProductSpecCharacteristicGroup2TmaProductSpecCharacteristicRelation.productSpecCharacteristics	Any side of a relation that has cardinality='many' should not have ordered='true' unless absolutely necessary
mtmcore- items.xml	115	ListsInRelationShouldBeAvoided	L	Customer2MtmCustomer.mtmCustomers	Any side of a relation that has cardinality='many' should have collectiontype='set' unless absolutely necessary
mtmcore- items.xml	126	ListsInRelationShouldBeAvoided	L	MtmTmaProductSpecCharacteristicGroup2TmaProductSpecCharacteristicRelation.productSpecCharacteristics	Any side of a relation that has cardinality='many' should have collectiontype='set' unless absolutely necessary
mtmcore- items.xml	146	ListsInRelationShouldBeAvoided	L	TmaSubscriptionBase2MtmDeviceInfo.devices	Any side of a relation that has cardinality='many' should have collectiontype='set' unless absolutely necessary

CR-03

17 medium and 5 low priority issues with the type system found. It's recommended to review and fix those problems before going live.

Type System and Relational Data Base Management System (RDBMS)

From HCS perspective the cloud solution for CCv2 uses Microsoft SQL-Server and some attributes in the data model has to be declared database specific.

In the help page [Specifying a Deployment for Commerce Platform Types](#) under section 'Advanced Deployment' you will find further information's and the standard mapping of HYBRIS.LONG_STRING to the dedicated RDBMS data types and the hint to core-advanced-deployment.xml in the 'Advanced Deployment' section. This file could differ for each new release and the below mapping is taken from release 6.6, 1808 and 1811.

REMARKS:

No hint detected in the release notes or the [Specifying a Deployment for Commerce Platform Types](#) page of the HYBRIS.JSON data type. This data type was added in 6.6 and so no need to change the data type of HYBRIS.LONG_STRING exist anymore. Instead of changing the persistence type database depended, the use of HYBRIS.JSON is recommended as best practice.

Data Type	RDBMS Data Type and Size				
	sap	oracle	hsqldb	mysql	sqlserver
java.lang.String	nvarchar(255)	varchar2(255 CHAR)	VARCHAR(255)	varchar(255)	nvarchar(255)
String	nvarchar(255)	varchar2(255 CHAR)	VARCHAR(255)	varchar(255)	nvarchar(255)
java.lang.Float	decimal(20,5)	number(20,5)	float	float(20,5)	float
java.lang.Double	decimal(30,8)	number(30,8)	double	double	float
java.lang.Byte	smallint	number(10,0)	smallint	smallint	integer
java.lang.Character	smallint	number(10,0)	smallint	smallint	char(4)
java.lang.Short	integer	number(10,0)	smallint	integer	integer
java.lang.Boolean	decimal(1,0)	number(1,0)	tinyint	tinyint(1)	tinyint
java.lang.Long	bigint	number(20,0)	bigint	bigint	bigint
java.lang.Integer	bigint	number(20,0)	int	integer	integer
float	decimal(20,5) DEFAULT 0	number(20,5) DEFAULT 0	float default 0	float(20,5) DEFAULT 0	float default 0
double	decimal(30,8) DEFAULT 0	number(30,8) DEFAULT 0	double default 0	double DEFAULT 0	float default 0
byte	integer DEFAULT 0	number(10,0) DEFAULT 0	smallint default 0	smallint DEFAULT 0	integer default 0
char	integer DEFAULT "	number(10,0) DEFAULT "	smallint default 0	smallint DEFAULT 0	char(4) default "
short	integer DEFAULT 0	number(10,0) DEFAULT 0	smallint default 0	integer DEFAULT 0	integer default 0
boolean	decimal(1,0) DEFAULT 0	number(1,0) DEFAULT 0	tinyint default 0	tinyint(1) DEFAULT 0	tinyint default 0
long	bigint DEFAULT 0	number(20,0) DEFAULT 0	bigint default 0	bigint DEFAULT 0	integer default 0
int	bigint DEFAULT 0	number(20,0) DEFAULT 0	int default 0	integer DEFAULT 0	integer default 0
java.util.Date	timestamp	timestamp(3)	timestamp	datetime	datetime2

java.math.BigDecimal	decimal(30,8)	number(30,8)	decimal(30,8)	decimal(30,8)	decimal(30,8)
java.io.Serializable	blob	blob	longvarbinary	longblob	image
HYBRIS.LONG_STRING	nvarchar(5000)	varchar2(4000)	longvarchar	text	nvarchar(max)
HYBRIS.JSON	nclob	clob	longvarchar	longtext	nvarchar(max)
HYBRIS.COMMA_SEPARATED_PKS	nvarchar(5000)	varchar2(4000)	longvarchar	text	nvarchar(max)
HYBRIS.PK	bigint	number(20,0)	BIGINT	bigint	bigint

As you can see here, the size for HYBRIS.LONG_STRING differs from RDBMS to RDBMS from 4000 to unlimited, which could cause problems changing from local development to other systems, because data could be truncated.

Actual Findings

We found **5** entries in the custom data model for persistence type HYBRIS.LONG_STRING with changed column type(s). It's valid to use HYBRIS.LONG_STRING but following best practice, every change of the column type should be avoided.

File	Starting Line	Type	Element	HYBRIS.LONG_STRING	sap	oracle	hsqldb	mysql	sqlserver	Status
mtmcore-items.xml	288	MtmRegistration	password	YES	NO	NO	NO	NO	NO	OK
mtmcore-items.xml	885	MtmSecurityCode	securityCode	YES	NO	NO	NO	NO	NO	OK
mtmcore-items.xml	758	PaymentTransactionEntry	response	YES	NO	NO	NO	NO	NO	OK
mtmcore-items.xml	775	CreditCardPaymentInfo	tokenNumber	YES	NO	NO	NO	NO	NO	OK
mtmintegration-items.xml	39	DatabaseLogEntry	message	YES	YES	NO	NO	YES	NO	NOK

CR-04

Fix the issue with the data type HYBRIS_LONG_STRING found in the custom data model. Set database accordingly or use the HYBRIS.JSON data type, following best practice.
mtmintegration-items.xml, type: DatabaseLogEntry, element: message

Search Restrictions

CR-05

The defined Search Restrictions are inline with SAP CX Field Services recommended practices.

Third-Party Libraries

The following third-party libraries not provided by the SAP Commerce are used in the project code:

Extension	LibraryName	RelativePath	Name	Version	OtherVersions
mtmcommercewebservices	javax.servlet.jsp.jstl	./web/webroot/WEB-INF/lib/	javax.servlet.jsp.jstl	1.2.5	mtmcommercewebservices:1.2.5;mtmmockserver:1.2.5
mtmcommercewebservices	stax-utils-20070216	./web/webroot/WEB-INF/lib/	stax-utils-20070216		mtmcommercewebservices;mtmmockserver:
mtmintegration	disruptor	./lib/	disruptor	3.3.4	mtmintegration:3.3.4
mtmintegration	jackson-datatype-jsr310	./lib/	jackson-datatype-jsr310	2.11.3	mtmintegration:2.11.3
mtmintegration	okhttp	./lib/	okhttp	3.9.1	mtmintegration:3.9.1
mtmintegration	okio	./lib/	okio	1.13.0	mtmintegration:1.13.0
mtmmockserver	javax.servlet.jsp.jstl	./web/webroot/WEB-INF/lib/	javax.servlet.jsp.jstl	1.2.5	mtmcommercewebservices:1.2.5;mtmmockserver:1.2.5
mtmmockserver	stax-utils-20070216	./web/webroot/WEB-INF/lib/	stax-utils-20070216		mtmcommercewebservices;mtmmockserver:

CR-06

The used third-party libraries are inline with SAP CX Field Services recommended practices.

Build Customization

During the deployment the customize folder is used to overwrite default platform behaviour directly in the platforms source instead of using custom extensions (preferred solution).

CR-07

No build customizations have been used / The defined Build Customizations are in line with SAP CX Field Services recommended practices.

INTEGRATION OVERVIEW

The following information about integration's were provided:

#	Source System	Target System	Purpose	Protocol
1	SAP Commerce	MTEL middle layer	To provide us necessary data which should be displayed to the customers (resources, bills, debt, etc), and also providing different web services which serve customer's online needs (change tariff, activate/deactivate addons, credit top up, pay debts, etc).	REST
2	SAP Commerce	SMS Center	It is used for sending sms to customers and also for calling some mtel's services which can be triggered by sending sms.	REST
3	SAP Commerce	Bank / Margento	It is used for different kinds of payment, manual payment of single or multiple bills by redirecting to WSPay portal where card info is being provided, also paying with a token stored on our portal by tokenizing the card, and also continuous automatic paying debt once a month for customers who activated standing order.	REST
4	Origami	SAP Commerce	To provide headless commerce functionality to the storefront application (Origami).	REST

NFRs

Daily average is around 7 transactions (orders).

Daily average is around 8k customers visiting the portal.

Daily average of concurrent customers is around 30, while in the peak it is increasing up to 60.

Page view per second, and in peaks for web and apps:

- web - per minute in peak is 24.7 and per second is 0.4 users (Black Friday), per day in peak we could expect around 1.4k per hour and 13k per day
- mobile app - per day 2.3 users, per minute 7 users, per sec 0,11 user

IO-01

From the provided information it's hard to make any conclusion. SAP Commerce is used as pure backend system which provides Commerce functionality via OCC to an external storefront application. As backend system, it calls telco's middleware layer to obtain certain data needed for the storefront as well as to trigger some events like change of the tariffs etc. Such use case is definitely supported and widely adopted by several SAP customers.

SUMMARY OF FINDINGS

All of the above-mentioned recommendations are summarized here. It is strongly recommended that MTel addresses these issues and implements or plans the implementation of the recommendations.

Conventions

Each of the recommendations has a:

1. **Unique key:** the key's prefix indicates the area of the recommendation.
2. **Short description:** describes the purpose of the recommendation and how to address it.
3. **Priority:** LOW, MED, or HIGH.
4. **Level of Effort:** LOW, MED, or HIGH.

The priority is indicated by the recommendation's colored box.

Assertion:

An assertion recognizes items that follow SAP CX Field Services recommended practices. No action required.

Improvement:

This low priority recommendation should be reviewed and worked into a longer-term plan, as it may have impact on future growth and performance.

Recommendation:

This recommendation should be reviewed and factored into upcoming releases as feasible.

Issue:

Issues should be reviewed and designated as a priority in the short term.

Below is a rough estimate of how much time would be required to correct the associated issue. Note that this is just an estimate and not an official guide for how long these items will take.

Level of Effort	Description
HIGH	High effort items will typically take more than a week.
MED	Medium effort items can typically be resolved in a week or less.
LOW	Low effort items can typically be resolved in a couple of days or less.

List of Recommendations

The following tables summarize the recommendations in each area covered in this review. The level of effort is simply a rough estimate and should be further investigated to get a finer estimate. This information is intended to assist in project prioritization.

Project Setup

Key	Category	Description	Priority	Level of Effort
PS-01	SAP Commerce Patch Level version Compatibility Check	Project is using SAP Commerce version 2005 with the auto-patching feature enabled.	✓	✓
PS-02	SAP Commerce Patch Level version Compatibility check	The current SAP Commerce SOLR setup is supported by SAP Commerce Cloud and is in line with the used SAP Commerce compatibility matrix. Solr version deployed is 8.4.1.	✓	✓
PS-03	Commerce Cloud Default Configuration	tomcat.generaloptions property is explicitly declared in common/local.properties file. This is the managed property in cloud and during the build process is overwritten.	MED	LOW
PS-04	Extensions and Dependencies	ldap extension is marked as deprecated since Commerce version 6.1. At the moment there is no suitable replacement available hence this extension remains in 2005 SAP commerce version.	N/A	HIGH
PS-05	Configuration settings	Current configuration settings are correct and in line with best practices.	✓	✓
PS-06	Source Control	There is an SCM in use, with proper configuration and usage. SCM contains detailed instructions on how to build the local instance. https://bitbucket.org/Wireless_Media/mtelme-hybris/src/dev/	✓	✓
PS-07	Deployment Package	Few errors while importing impex files during initialization: 1) [ImpExSystemSetup] importing resource : /impex/essentialdataAdhocWorkflowLocalizations_XX.impex 2) [ImpExSystemSetup] importing resource : /customersupportbackoffice/impex/essentialdata-customersupportgroups_XX.impex Exception when Executing indexer worker: ERROR: [doc=8796099608577] unknown field '_backofficeLabel_text_me'	MED	LOW
PS-08	Storefront compression with Wro4j	WRO4J storefront compression is not enabled. But that is irrelevant as SAP Commerce is used in headless mode.	N/A	N/A
PS-09	Cloud Portal Configurations	Backoffice, API and Solr endpoints are exposed to public internet. It's recommended to use IP filter instead to avoid security risks.	HIGH	LOW
PS-10	Cloud Portal Configurations	This review identified that the manifest.json definition is correct and contains all of the expected configurations. commerceSuiteVersion is configured to use the latest patch instead of explicitly defining a patch. Configurations have been reused to make local development as close to what will be deployed in production.	✓	✓

Catalog Design Review

Key	Category	Description	Priority	Level of Effort
CD-01	Product Catalog Design	The current catalog design is implemented based on SAP Commerce catalog design #1 (Single Product Catalog) but following the data model from Telco accelerator.	✓	✓
CD-02	Product Variants	Product variants are made following SAP CX Field Services recommended practices.	✓	✓
CD-03	Classification System	The Classification System is not used.	✓	✓
CD-04	Content Catalog Design	Standard one content catalog approach with two catalog versions. MTel is using SAP Commerce in headless mode.	✓	✓
CD-05	Multi-Tenancy	The Multi-tenancy mode is inactive and follows the SAP CX Field Services recommended practices.	✓	✓

Static Source Code Analysis

Key	Category	Description	Priority	Level of Effort
CR-01	Source Code Analysis	Only two bugs (1 critical, 1 major) were found by the Sonarqube.	HIGH	LOW
CR-02	Code Coverage	It was not possible to gather code coverage information due to technical issue with the setup.	N/A	N/A
CR-03	Type System	17 medium and 5 low priority issues with the type system found. It's recommended to review and fix those problems before going live.	HIGH	MED
CR-04	Type System and RDBMS	Fix the issue with the data type HYBRIS_LONG_STRING found in the custom data model. Set database accordingly or use the HYBRIS.JSON data type, following best practice. mtmintegration-items.xml, type: DatabaseLogEntry, element: message	MED	LOW
CR-05	Search Restrictions	The defined Search Restrictions are inline with SAP CX Field Services recommended practices.	✓	✓
CR-06	Third-Party Libraries	The used third-party libraries are inline with SAP CX Field Services recommended practices.	✓	✓
CR-07	Build Customization	No build customizations have been used / The defined Build Customizations are in line with SAP CX Field Services recommended practices.	✓	✓

Integration Overview

Key	Description	Priority	Level of Effort
IO-01	From the provided information it's hard to make any conclusion. SAP Commerce is used as pure backend system which provides Commerce functionality via OCC to an external storefront application. As backend system, it calls telco's middleware layer to obtain certain data needed for the storefront as well as to trigger some events like change of the tariffs etc. Such use case is definitely supported and widely adopted by several SAP customers.	✓	✓

www.sap.com/contactsap

© 2021 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

The information contained herein may be changed without prior notice. Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP Customer Experience

THE BEST RUN

